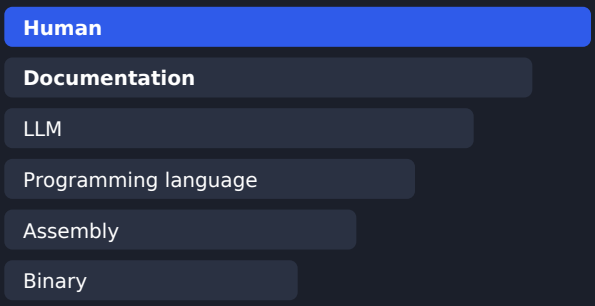
STANDARD · VERSION 1.1 · OCTOBER 2026

Doc-First Engineering Standard

Humans, LLMs and the future of software work: principles, roles and documentation standards.

Olwethu Nguza
Technical Lead (ex-FAANG)



Version History

Version	Date	Summary
0.1	September 2026	Initial draft.
1.1	October 2026	Consolidated. Adds infrastructure layout and Terraform rules, hyperlinking, proactive runbooks, environment guides and two new templates. Aligns normative language and removes duplicated rules.

Contents

1. Purpose and Scope
2. The Abstraction Ladder
3. Core Thesis
4. Roles and Responsibilities
5. Principles
6. The Documentation Set
7. Repository Layout
8. ADR Standard
9. Writing for Humans and LLMs
10. Hyperlinking and Traceability
11. The Change Lifecycle

12. Bugs and Root Cause
13. Runbooks and Proactive Operations
14. Review Standard
15. The Human Skill Shift
16. Risks and Limits
17. Adoption Levels
18. Glossary
19. Appendices: Templates

How to read this. Sections 1 to 5 set the vision. Sections 6 to 14 are the working standard. Sections 15 to 17 cover people, risk and adoption. Templates are at the end.

SECTION 01

Purpose and Scope

This standard defines how software is built when Large Language Models (LLMs) write most of the code. It sets out what humans own, what LLMs own, and how documentation becomes the primary artifact that engineers create, review and maintain.

Applies To

- New systems designed with LLM-generated implementation from the start.
- Existing systems moving towards documentation-led development.
- Application code and infrastructure as code alike.
- Engineers, architects, testers, product teams and operations teams.

Does Not Cover

- Selection of LLM vendors or tools.
- Automated enforcement such as CI/CD gates. This standard relies on human discipline and review. Automation MAY be added later without changing the principles.
- Model training or fine-tuning.

Standards Language

- **MUST** means mandatory.
- **MUST NOT** means prohibited.
- **SHOULD** means expected unless there is a documented reason not to.
- **MAY** means optional.

These words are written in capitals wherever they carry this meaning.

SECTION 02

The Abstraction Ladder

Every major shift in software has moved humans one layer further from the machine. Each layer lets people express more intent with less detail.

Layer	What Humans Write
Binary	Machine instructions as raw bits
Assembly	Named instructions and registers
Programming Language	Logic, data structures and abstractions
LLM	Prompts that produce code
Documentation	Durable intent: architecture, decisions, rules and acceptance criteria
Human	Judgment about what should be built and whether it is correct

What Is Different This Time

Every layer below the LLM is deterministic. A compiler turns identical source code into identical output. An LLM does not. The same documentation may produce several different, equally plausible implementations.

> **Consequence.** Because generation is non-deterministic, verification becomes a first-class engineering activity. Documentation defines what is correct. Tests and review confirm it.

Documentation Is Not a Prompt

A prompt is disposable and conversational. Documentation is versioned, reviewed, structured and owned, in the same way source code always has been.

SECTION 03

Core Thesis

> **Code becomes a derived artifact. Documentation becomes the source.** Human engineering work moves up a layer, to defining intent, making decisions and judging results.

Claim 1: Documentation Is the Source of Truth

When code and documentation disagree, documentation is assumed correct until a human decides otherwise. The disagreement itself is a defect, and it MUST be resolved in one direction or the other.

Claim 2: The Mapping Is Layered, Not Line by Line

A strict one-to-one mapping between every line of code and every line of documentation is neither possible nor useful. Natural language underspecifies, and many programs satisfy one description. The mapping that works is:

```
Decision / Rule
       ↕
Implementation
       ↕
Test or Validation
```

Every decision and rule has a home in the implementation (a domain module or a Terraform module) and a way to confirm it. Detail below that level belongs to the LLM and is judged by behaviour, not wording. Section 10 sets out the full navigation chain.

Claim 3: Reasoning Must Survive

Code shows what a system does. Only documentation shows why. When code is regenerated or refactored, the reasoning MUST remain intact, because it is what lets the next person, or the next model, make a consistent change.

SECTION 04

Roles and Responsibilities

Humans own the why and the what. LLMs own the how.

Party	Owns
Human	Problem framing, scope, architecture, trade-offs, constraints, acceptance criteria, security posture, approval and accountability

Party	Owns
LLM	Implementation, infrastructure code, boilerplate, refactoring, test scaffolding, first-pass review, first-pass root-cause analysis, and diagnostic commands drafted from runbooks
Shared	Review. The LLM checks code against documentation. The human checks that documentation still describes what the business needs, and that the code meets it

Human Roles

Role	Focus
Architect	System shape, boundaries, ADRs, cross-cutting rules
Domain Owner	Business rules and invariants for one domain
Platform Owner	Terraform modules, environments, infrastructure standards
Specifier	Precise, testable requirements and acceptance criteria
Reviewer	Verifies code and infrastructure against documentation
Operator	Environment guides, runbooks, incidents and feeding production learning back into documentation

One person MAY hold several roles. Every responsibility MUST have a named owner.

Accountability

> An LLM can write code. It cannot own a decision. Every merged change MUST have a human approver who can explain why it is correct.

SECTION 05

Principles

P1. Documentation Is the Source of Truth. If code and documentation disagree, documentation wins until a human decides otherwise.

In practice: resolve every disagreement explicitly. Either fix the code, or update the documentation and record why.

P2. Change Starts in Documentation. Code without matching documentation is unowned.

In practice: update the relevant document first, then generate or change code (Section 11).

P3. Record Decisions, Not Just Descriptions. The why outlives the what.

In practice: any real choice between alternatives gets an ADR (Section 8).

P4. Every Rule Is Verifiable. A rule nobody can check is a wish.

In practice: write concrete conditions, examples and acceptance criteria that a person can confirm.

P5. Structure Is Consistent. Humans and LLMs navigate by pattern.

In practice: every domain, module and environment follows the same layout (Section 7).

P6. Every Bug Is Classified. The type of failure decides the fix.

In practice: classify as Decision Wrong, Doc Missing, Code Diverged or Environment (Section 12).

P7. Archive, Do Not Delete. Superseded reasoning is still evidence.

In practice: move replaced documents to `docs/archive/`, mark them superseded and link to their replacement.

P8. Humans Stay Accountable. LLM speed does not transfer responsibility.

In practice: named human approval on every change, and a second reviewer for sensitive changes (Section 14).

P9. One Fact, One Place. Duplicated facts drift apart.

In practice: each rule, value or decision lives in exactly one document. Others link to it (Section 10).

P10. Documentation Has Two Readers. Humans and LLMs both consume it.

In practice: plain, precise language, defined terms, explicit constraints, examples over adjectives (Section 9).

P11. Prepare Before It Breaks. *We cannot see the future, but we can prepare for it.*

In practice: before a component goes live, document how it can fail, how the failure shows itself, and what to run first. Runbooks are written proactively, not after the first incident (Section 13).

SECTION 06

The Documentation Set

Every system SHOULD maintain the following documents. Small systems MAY merge them, but each type of content still needs a home.

Context

Purpose: Why the system exists, users, scope, key terms
Owner: Product / Architect
Update On: Scope changes

Architecture

Purpose: System shape, domains, boundaries, integrations, data flow
Owner: Architect
Update On: Structural changes

Infrastructure Architecture

Purpose: Cloud topology, networking, environments, one section per capability
Owner: Architect / Platform Owner
Update On: Infrastructure changes

ADRs

Purpose: One record per significant decision
Owner: Architect / Decision Maker
Update On: Any real choice between alternatives

Domain Rules

Purpose: Invariants, business rules, edge cases, acceptance criteria

Owner: Domain Owner

Update On: Any behaviour change

Contracts

Purpose: API specifications and data schemas

Owner: Domain Owner

Update On: Any interface change

Coding Standards

Purpose: Layering, naming, errors, logging, security conventions

Owner: Architect

Update On: Convention changes

Infrastructure Standards

Purpose: Terraform conventions, module design, tagging, naming

Owner: Platform Owner

Update On: Convention changes

Testing Expectations

Purpose: What "done" means per domain and per module

Owner: Domain Owner / Tester

Update On: New rules, domains or modules

Security

Purpose: Identity, authorisation, secrets handling, data classification

Owner: Security Owner

Update On: Any security-relevant change

Operations Standard

Purpose: What must be true in operation: severity levels, on-call, change windows. Links to runbooks for procedures

Owner: Operator

Update On: Process changes

Environment Guides

Purpose: One per environment: accounts, regions, profiles, contexts, resource names

Owner: Operator / Platform Owner

Update On: Any environment change

Runbook

Purpose: Known failure modes, procedures, diagnostic commands

Owner: Operator

Update On: Before go-live, after incidents and near misses

Changelog

Purpose: What changed, when, and which documents it touched

Owner: Everyone

Update On: Every change

The Operations Standard says **what** must happen. The Runbook says **how**. Environment Guides hold the **facts** both depend on.

SECTION 07

Repository Layout

Documentation lives in the same repository as code and infrastructure, is versioned with it, and changes in the same commit. It is organised by audience, so each reader finds their material in one place.

```
.
├── docs/
│   ├── README.md
│   ├── CHANGELOG.md
│   ├── context/
│   │   ├── overview.md
│   │   └── glossary.md
│   ├── architecture/
│   │   ├── architecture.md
│   │   ├── infrastructure.md
│   │   ├── security.md
│   │   └── decision-records/
│   │       └── ADR-001-<title>.md
│   ├── domains/
│   │   └── <domain>/
│   │       ├── rules.md
│   │       └── acceptance.md
│   ├── api/
│   │   ├── specification.md
│   │   └── openapi.yaml
│   ├── database/
│   │   └── conventions.md
│   ├── standards/
│   │   ├── coding.md
│   │   ├── infrastructure.md
│   │   └── testing.md
│   ├── operations/
│   │   ├── operations-standard.md
│   │   ├── runbook.md
│   │   ├── environments/
│   │   │   └── <environment>.md
│   │   └── incidents/
│   └── archive/
├── infrastructure/
│   ├── terraform/
│   │   ├── environments/
│   │   │   └── <environment>/
│   │   │       └── README.md
```

```
│   │   └── modules/
│   │       └── <module>/
│   │           └── README.md
│   └── docker/
│       ├── backend.Dockerfile
│       └── frontend.Dockerfile
└── src/
    └── <domain>/
        ├── api/
        ├── application/
        ├── domain/
        └── infrastructure/
```

Typical environments are `dev`, `sandbox`, `sit`, `staging` and `production`. Typical modules include `network`, `eks`, `eks-addons`, `rds`, `alb`, `nlb`, `api-gateway`, `waf`, `kms`, `secrets`, `ecr`, `cloudwatch`, `s3-audit` and `gitlab-ci-iam`. The set varies by system; the structure does not.

Code Layering

Every domain under `src/` uses the same four layers:

Layer	Holds
`api/`	Controllers, DTOs
`application/`	Use cases
`domain/`	Entities, rules
`infrastructure/`	Repositories, adapters

Domains MUST NOT reach into another domain's `domain/` or `infrastructure/` layers. Where they must interact, they use a published interface or event.

Infrastructure Structure Rules

- `infrastructure/terraform/environments/<environment>/` MUST contain only environment-specific composition, configuration and variable values.
- Shared infrastructure logic MUST be implemented in `infrastructure/terraform/modules/`.
- Environment directories MUST consume reusable modules and MUST NOT duplicate module implementation. Environments differ by variable, not by copied code.
- Each module and each environment directory SHOULD contain a `README.md`. A module README describes purpose, inputs, outputs and dependencies. An environment README lists the modules it instantiates.
- Docker build definitions MUST be stored in `infrastructure/docker/`.
- Operational procedures MUST be documented in the runbook, not left as undocumented manual steps beside Terraform files.
- Infrastructure decisions that need an ADR are defined in Section 8.

Code Mirrors Documentation

> `docs/domains/<domain>/`
>
> Implementation: `src/<domain>/`

> **Capability section in** `docs/architecture/infrastructure.md`
>
> Implementation: `infrastructure/terraform/modules/<module>/`

> `docs/operations/environments/<environment>.md`
>
> Implementation: `infrastructure/terraform/environments/<environment>/`

A reader, human or LLM, MUST be able to move from documentation to implementation without relying on tribal knowledge.

SECTION 08

ADR Standard

Architecture Decision Records (ADRs) make reasoning recoverable, not just outcomes. They are the backbone of this standard.

When an ADR Is Required

- Any real choice between alternatives in application design.
- Any infrastructure change that affects architecture, security, availability, cost, networking or operational ownership.

Required Sections

Section	Contains
Status	Proposed, Accepted, Superseded by ADR-XXX, or Deprecated
Context	Problem, constraints, risks
Decision	What was decided, in one or two sentences
Alternatives	Options considered and why each was rejected
Consequences	What gets easier, what gets harder
Rules	Concrete MUST / MUST NOT rules the implementation follows
Review Trigger	Conditions under which the decision is reopened
References	Hyperlinks to affected documents, modules, environments and tests

ADR Rules

- ADRs are numbered sequentially. Numbers are never reused or renumbered. A skipped number is recorded, not filled.
- Accepted ADRs MUST NOT be substantively modified. A changed decision gets a new ADR, and the old ADR's status is updated to point to it.
- References MUST use repository-relative links to modules, classes or files, never line numbers (Section 10).

- Infrastructure ADRs MUST link to the affected Terraform modules and environments.
- An ADR SHOULD fit on a few phone screens. If it is longer, it probably holds more than one decision.

> **Why the review trigger matters.** It makes a decision conditional rather than permanent. Before proposing a change that contradicts an ADR, a human or LLM checks whether its trigger has fired.

SECTION 09

Writing for Humans and LLMs

Documentation has two readers. What helps an LLM generate correct code also helps a human understand the system. The standard for both is precision.

Do

- Use **MUST**, **SHOULD** and **MAY** so the strength of each rule is unambiguous.
- Give every rule a **Rule ID** (for example `ALLOC-003`) so code, tests and bug reports can refer to it.
- Define terms once in the glossary and use them consistently. One concept, one word.

- Prefer **examples** over adjectives. Show a valid and an invalid input rather than saying "well-formed".
- State **negative rules** explicitly. What MUST NOT happen often matters more than what should.
- Include **edge cases**: empty, duplicate, concurrent, expired, unauthorised.
- Write **exact names** in environment guides and runbooks: resources, regions, profiles, contexts.
- Use repository-relative links for every reference to another document or artifact.
- Keep sentences short, one idea each.

Avoid

- Vague words such as fast, secure, appropriate and as needed. Replace them with numbers or conditions.
- Restating a rule in several places (P9).
- Implementation detail the LLM should decide, unless it is a genuine constraint.
- Narrative history in active documents. History belongs in ADRs, the changelog or the archive.
- Local file-system paths or line numbers.

Example

Weak

```
Allocation should handle duplicates properly.
```

Strong

```
ALLOC-003:
A repeated request with the same idempotency key
MUST return the original result and MUST NOT
create a new record.

A repeated key with a different request body
MUST be rejected with a conflict error.
```

SECTION 10

Hyperlinking and Traceability

Hyperlinking

Hyperlinking is embedding a clickable link in a document that takes the reader directly to another document, section or implementation artifact. It is how this standard applies P9: facts live in one place, and everything else links to them.

Hyperlinking Rules

- Documentation MUST use repository-relative Markdown links for content in the same repository.
- Link text MUST describe the destination. Generic text such as "click here", "this document" or "more information" MUST NOT be used.
- ADRs MUST link to the affected architecture documents, modules and environments.
- Module READMEs SHOULD link back to the ADRs, standards and architecture sections that govern them.
- Environment READMEs SHOULD link to the modules they instantiate and to their environment guide.
- The Operations Standard SHOULD link to the runbook for procedures, commands and troubleshooting.
- Links MUST be updated in the same change when files or directories move.
- A broken link is a documentation defect.

Format

```
[Descriptive link
  text](relative/path/to/file.md#section)
```

Paths are relative to the file that contains the link. Count the `../` steps from that file.

Examples

From an ADR in `docs/architecture/decision-records/` to a module:

```
Implemented by the
[Network Terraform Module](../../../infrastructure/te
  rraform/modules/network/README.md).
```

From `docs/architecture/infrastructure.md` to an environment:

```
See the
[Sandbox Environment Configuration](../../infrastruct
  ure/terraform/environments/sandbox/README.md).
```

From `docs/operations/operations-standard.md` to a runbook section:

```
See [Database Capacity
  Response](runbook.md#database-capacity-response).
```

Capability Sections

Each infrastructure capability SHOULD have a section in `docs/architecture/infrastructure.md` that links to everything about it. For example:

```
## Network

- [Decision
  Record](decision-records/ADR-012-network-design.md)
- [Terraform Module](../../infrastructure/terraform/m
  odules/network/README.md)
- [Dev Environment](../../infrastructure/terraform/en
  vironments/dev/README.md)
- [SIT Environment](../../infrastructure/terraform/en
  vironments/sit/README.md)
- [Production Environment](../../infrastructure/terra
  form/environments/production/README.md)
- [Operations
  Standard](../operations/operations-standard.md)
- [Runbook: Network Operations](../operations/runbook
  .md#network-operations)
```

Traceability Chain

Traceability means any rule can be followed to the implementation and test that honour it, and any implementation can be followed back to the rule it serves. Links SHOULD let a reader walk this chain:

```
Decision or Rule (ADR, domain rules)
        ↓
Architecture Documentation
        ↓
Implementation (domain module or Terraform module)
        ↓
Environment Configuration (infrastructure only)
        ↓
Test or Validation
        ↓
Operations Standard and Runbook
```

Maintaining It Without Automation

- Rule IDs appear in test names or descriptions.
- Each domain's `rules.md` lists its Rule IDs and the module that owns each.
- Code comments cite a Rule ID only where the reason is not obvious. They never restate the rule.
- Reviewers check that new or changed rules have a matching test, and that removed rules have their tests removed.

The purpose is navigation, not a separate mapping register. Link to the authoritative artifact wherever practical.

> **The test is the executable form of the documentation.** Where a rule matters, a test is the only way to confirm that a regenerated implementation still honours it.

SECTION 11

The Change Lifecycle

Every change, from a new feature to a one-line fix or a Terraform variable, follows the same five steps. Small changes move through them quickly. The order does not change.

1. **Define.** A human writes or updates the requirement, rule or acceptance criteria.
2. **Decide.** If the change involves a real choice, a human records it in an ADR or confirms an existing ADR still holds.
3. **Generate.** The LLM implements the change using the documentation as context and following the standards.
4. **Verify.** Tests and validation run. The LLM checks the change against the documentation. A human reviews it (Section 14).
5. **Reconcile.** Any gap found is fixed in the documentation or the implementation. Links and the changelog are updated, and the change is merged.

The Rule

> Every code or infrastructure change MUST include a documentation update, or an explicit statement: *No doc impact, because...* The reason is what keeps the statement honest.

When a change moves files, adds or removes environments or modules, or changes controls or ownership, the affected links MUST be updated in the same change.

When Code Leads

During an incident or an experiment, code MAY change first. The change is not complete until the documentation is reconciled. Record the reconciliation as a follow-up with an owner, not as a memory.

SECTION 12

Bugs and Root Cause

In a doc-first system, documentation is the reference point for diagnosis. The first question is not "what broke?" but "which layer is wrong?"

Decision Wrong

Meaning: Documentation followed correctly, but the decision was wrong

Fix: New superseding ADR, then documentation and code

Doc Missing

Meaning: The situation was never specified, so the LLM guessed

Fix: Add the rule and edge case, then fix the code

Code Diverged

Meaning: Documentation correct, implementation does not follow it

Fix: Fix the implementation; add the missing test

Environment

Meaning: Configuration, data, infrastructure or a dependency

Fix: Fix the environment or module; update the environment guide and runbook

Root-Cause Protocol

1. **Reproduce** the bug and gather evidence: logs, traces, request IDs, data state.
2. **Ground** the LLM with the relevant documentation and evidence. Ask which rules apply and whether the observed behaviour violates them.
3. **Classify** using the table above. A human confirms the classification.
4. **Fix** at the correct layer: decision, documentation, implementation or environment.
5. **Prevent** recurrence by adding the test or validation that would have caught it.
6. **Update** affected documentation and links.
7. **Record** incidents and near misses in `docs/operations/incidents/` with the classification.

> **Limit.** Documentation describes intended behaviour. It cannot show runtime state, concurrency, data quality or infrastructure faults. Evidence from the running system is always required.

SECTION 13

Runbooks and Proactive Operations

We cannot see the future, but we can prepare for it.

Infrastructure and incident work is where doc-first pays off fastest. The runbook and environment guides do two jobs:

- **Before an incident**, they hold the failure modes and diagnostic commands, written in advance.
- **During an incident**, they are the LLM's source of truth. The LLM turns them into exact, ready-to-run commands.

The shift. The human no longer types or looks up commands. The human describes the symptom, supplies the documentation, decides what to run and judges the results. The LLM supplies the full command with the correct profile, region, context and resource names, taken from the documentation.

Example

An admin console in SIT is returning errors. The engineer gives the LLM the SIT environment guide, the runbook and the symptom. The LLM proposes read-only checks using names from the guide (names here are illustrative):

```
# 1. Is the application running?
kubectl --context sit -n app get pods

kubectl --context sit -n app \
  logs deploy/app-api --since=30m \
  | grep -i error

# 2. Is the database healthy?
aws rds describe-db-instances \
  --profile app-sit \
  --region <region-from-guide> \
  --db-instance-identifier app-sit \
  --query \
  'DBInstances[0].DBInstanceStatus'

# 3. Did the secrets sync?
kubectl --context sit -n app \
  get externalsecrets
```

The engineer runs them and pastes the output back. The LLM narrows the cause against the runbook's known failure modes, for example a database credential rotated upstream but not yet synced.

Investigation Loop

1. **Describe.** The human states the symptom, environment and time window.
2. **Ground.** The human supplies the runbook, environment guide and relevant ADRs.
3. **Propose.** The LLM suggests read-only commands, each with a one-line reason.
4. **Run.** The human executes them and returns the output.
5. **Narrow.** The LLM compares evidence with documented behaviour and known failure modes, then proposes the next step.
6. **Decide.** The human chooses the fix. Any change to an environment is the human's decision.
7. **Record.** The runbook is updated with any new failure mode and the commands that found it.

Environment Guide Requirements

An LLM can only produce exact commands if the documentation contains exact facts. Each environment guide MUST include (template in Appendix F):

- Account, region, CLI profile, cluster context and namespaces.
- Resource names: databases, deployments, log groups, load balancers, queues.

- Secret **names and locations, never values**, and how each syncs.
- Dependencies: what calls what, so the LLM knows where to look next.
- Links to the environment's Terraform directory and the runbook.
- Which operations are safe (read-only) and which change state.

Proactive by Design

Every component, module and integration SHOULD answer three questions **before** it goes live:

1. **How can this fail?** Outages, expired credentials, full disks, exhausted connections, bad deployments, upstream changes, quota limits.
2. **How will we know?** The symptom, alarm, log line or user report.
3. **What do we run first?** The read-only commands that confirm or rule out the cause, and the documented fix.

The answers go into the runbook as failure mode entries (template in Appendix E).

Using the LLM to Prepare

- **Pre-mortem.** Give the LLM the architecture documentation and ADRs and ask how each component could fail. The human keeps the realistic failure modes and discards the rest.
- **Prepared commands.** For each failure mode, the LLM drafts diagnostic commands from the environment guide. A human checks them in a non-production environment.
- **New decisions, new failure modes.** Every new ADR, module or component adds its failure modes in the same change, not after the first incident.
- **Drills.** Periodically walk through a failure mode in SIT or sandbox with the runbook and the LLM, to confirm the commands and names are still correct.

Near misses count. Anything that almost failed, or failed quietly, is recorded the same way as an incident. Preparing for the next failure is cheaper than investigating it.

> **Reactive:** something breaks, then people work out what to run.
> **Proactive:** something breaks, and the runbook already says what to run.

Safety Rules

- **Read before write.** Investigation starts with read-only commands: describe, get, logs, list.
- **Label mutating commands.** Anything that restarts, scales, deletes, rotates or modifies MUST be marked as a change and explained before it runs.
- **Humans execute.** The LLM proposes. It MUST NOT run commands against shared environments.
- **Explicit context.** Every command MUST name its profile, region and cluster context. Never rely on whatever the shell happens to point at.
- **No invented names.** If a resource name is not in the documentation, the LLM MUST ask rather than guess. A guessed name that happens to exist is worse than one that fails.
- **No secrets in the conversation.** Redact credentials, tokens and personal data before sharing output.
- **Production is stricter.** Production changes follow the change process (Section 11), not the investigation loop.

SECTION 14

Review Standard

Generated code and infrastructure are reviewed as if written by a capable, fast colleague who has never met your users: usually correct, occasionally confidently wrong.

Every Reviewer Checks

- **Intent.** Does the change do what the updated documentation says, and only that?
- **Documentation.** Is it updated, or is "no doc impact" justified?
- **Rules.** Are affected Rule IDs covered by tests or validation?
- **Boundaries.** Does code stay within its domain and layer? Does infrastructure stay within its module?
- **Standards.** Naming, error handling, logging, Terraform conventions.
- **Traceability.** Are links present, correct and updated?
- **Environment impact.** Which environments change, and is the change confined to variables where it should be?
- **Surprises.** New dependencies, files outside the expected layout, or anything nobody asked for.

Second Human Review

Changes in these areas SHOULD have a second human reviewer:

- **Security:** authentication, authorisation, secrets, IAM permissions, encryption.
- **Data:** personal or regulated data, database migrations, anything irreversible.
- **Correctness:** financial or identity logic, concurrency, transactions, idempotency.
- **Infrastructure:** networking, public exposure, production infrastructure.

SECTION 15

The Human Skill Shift

When writing code becomes cheap, the scarce skills are the ones that decide what code should exist and whether it is right.

Declining in Value	Rising in Value
Syntax recall	Precise technical writing
Framework trivia	Systems and architecture thinking
Typing speed	Domain knowledge
Boilerplate	Edge-case specification
Solo implementation	Sceptical review and verification

Declining in Value	Rising in Value
Memorised commands	Operational judgment and diagnosis
Security mechanics	Security and risk judgment

What This Means for Engineers

- Reading code remains essential. You cannot review what you cannot read.
- Writing becomes an engineering skill, not an afterthought.
- Junior engineers need deliberate practice in fundamentals, because the LLM removes the repetition that used to teach them.
- Seniority is measured by the quality of decisions and specifications, not lines produced.

> The strongest engineers specify precisely and review sceptically.

SECTION 16

Risks and Limits

Non-determinism

What Happens: The same documentation produces different code

Mitigation: Tests as the executable contract; human review

Over-trust

What Happens: Plausible code is accepted without real review

Mitigation: Review standard; named accountability

Documentation drift

What Happens: Code changes while documentation lags

Mitigation: The change rule; reconcile step; reviewers check documents and links

Documentation bloat

What Happens: Documents grow until nobody reads them

Mitigation: One fact, one place; archive; short ADRs

Underspecification

What Happens: The LLM silently fills gaps with guesses

Mitigation: Edge cases in rules; Doc Missing bug class

Security blind spots

What Happens: Generated code introduces subtle vulnerabilities

Mitigation: Second human review

Skill atrophy

What Happens: Engineers lose the ability to reason about code

Mitigation: Deliberate practice; rotating deep reviews

Context limits

What Happens: The LLM cannot hold the whole documentation set

Mitigation: Consistent structure; focused per-domain documents

Reactive runbooks

What Happens: The first failure is always slow

Mitigation: Pre-mortems; failure modes documented before go-live

Wrong-environment commands

What Happens: A correct command runs against the wrong account or cluster

Mitigation: Explicit profile, region and context; read before write

Environment drift

What Happens: Environments diverge from each other

Mitigation: Reusable modules; environments differ by variable only

Module duplication

What Happens: Infrastructure logic is copied into environments

Mitigation: Platform Owner owns modules; review boundary check

SECTION 17

Adoption Levels

Teams adopt this standard in stages. Each level builds on the previous one.

Level 1: Documented. Context, architecture and standards exist in the repository.

Level 2: Decided. Significant decisions have ADRs in one format, with review triggers.

Level 3: Doc-Led. Every change updates documentation or states "no doc impact". Rules have IDs. Documents link to each other.

Level 4: Traceable. Rules link to code, modules, environments and tests. Bugs are classified. Runbooks drive investigations, and incidents feed back into documentation.

Level 5: Source of Truth. Failure modes are documented before go-live. Documentation is complete enough that any domain or module could be regenerated from it and verified by its tests.

> **Where to start.** Most teams reach Level 3 quickly: consistent ADRs, audience-based documents and a documentation update on every change. The usual next step is Rule IDs linked to tests.

SECTION 18

Glossary

ADR. Architecture Decision Record: a short document recording one decision, its alternatives and consequences.

Acceptance Criteria. Conditions a change must meet to be considered done.

Doc-First. Documentation is updated before or with code, and is the source of truth.

Drift. Documentation and implementation no longer describe the same behaviour.

Environment Composition. The Terraform configuration that selects and configures reusable modules for one environment.

Environment Guide. The document holding the exact facts of one environment: accounts, regions, contexts and resource names.

Failure Mode. A documented way a component can fail, with its symptom, likely cause, diagnostic commands and fix.

Hyperlinking. Embedding a clickable link that takes the reader to another document, section or artifact.

Invariant. A rule that must always hold, such as "audit events are never modified".

Near Miss. Something that almost failed, or failed without visible impact. Recorded like an incident.

Pre-mortem. Asking, before go-live, how a component could fail, so the runbook is ready in advance.

Review Trigger. A condition that, when met, means an ADR should be reconsidered.

Rule ID. A stable identifier for one rule, used in code, tests and bug reports.

Runbook. The document holding failure modes, procedures and diagnostic commands.

Source of Truth. The artifact that wins when two artifacts disagree.

Traceability. Being able to follow a rule to its implementation, environment and test, and back.

APPENDIX A

ADR Template

```
# ADR-NNN: <decision title>

Status: Proposed | Accepted |
        Superseded by ADR-XXX | Deprecated
Date: YYYY-MM-DD
Owner: <name>

## Context
<problem, forces, constraints>

## Decision
<one or two sentences>

## Alternatives
- <option>: rejected because <why>

## Consequences
- Easier: <...>
- Harder: <...>

## Rules
- <ID>: <MUST / MUST NOT rule>

## Review Trigger
<conditions to reopen>

## References
- Documentation: <Markdown links>
- Modules: <Markdown links>
- Environments: <Markdown links>
- Tests or validation: <names>
```

APPENDIX B

Domain Rules Template

```
# <Domain> Rules

## Responsibility
<what this domain owns and does not own>

## Invariants
- <ID>: <always true>

## Rules
- <ID>: <MUST / SHOULD rule>
  Valid:   <example>
  Invalid: <example>

## Edge Cases
- Empty / Duplicate / Concurrent
- Expired / Unauthorised

## Acceptance Criteria
- <condition for done>

## Rule Ownership
| Rule ID | Module | Test |

## Owner
<name>
```

APPENDIX C

Change Description Template

```
## Change
<what and why, one paragraph>

## Docs Updated
- <file>: <what changed>
  or: No doc impact, because <...>

## Rules Affected
- <ID>: added / changed / removed

## Decision
ADR-NNN still holds | New ADR-NNN

## Environments Affected
- <environment>: <what changes>

## Links
- Updated: Yes | Not applicable

## Verification
- Tests or validation: <names>
- Reviewer: <name>
- Second reviewer: <name, if required>
```

APPENDIX D

Bug Record Template

```
## Bug
<symptom and evidence>

## Environment and Time Window
<environment, start and end time>

## Classification
Decision Wrong | Doc Missing |
Code Diverged | Environment

## Rules Involved
- <ID>

## Commands Run
<read-only checks and results>

## Fix
<layer fixed and how>

## Test or Validation Added
<name>

## Documentation and Links Updated
<file and change, or not applicable>

## Runbook Updated
<failure mode added or changed>
```

APPENDIX E

Failure Mode Entry Template

One entry per known failure mode, in `docs/operations/runbook.md`.

```
## <Failure mode title>

Component: <domain, module or service>
Environments: <all, or list>

### Symptom
<what users, alarms or logs show>

### Likely Causes
- <cause>

### Diagnose (read-only)
<exact commands, with profile,
 region and context>

### Fix
<steps; label every mutating
 command as a change>

### Prevention
<test, alarm or design change>

### History
- <date>: <incident or near miss link>
```

APPENDIX F

Environment Guide Template

One per environment, in `docs/operations/environments/<environment>.md`.

```
# <Environment> Environment Guide

## Purpose
<what this environment is for>

## Access
- Account: <id or alias>
- Region: <region>
- CLI profile: <profile>
- Cluster context: <context>
- Namespaces: <list>

## Resources
| Type | Name |
| Database | <identifier> |
| Deployment | <name> |
| Log group | <name> |
| Load balancer | <name> |

## Secrets
| Name | Location | Sync method |
(names only, never values)

## Dependencies
<what calls what>

## Safe and Unsafe Operations
- Read-only: <commands>
- Changes state: <commands>

## Links
- [Terraform Environment](<relative path>)
- [Runbook](<relative path>)
```

End of standard. Version 1.1.

Doc-First Engineering Standard by Olwethu Nguza, https://github.com/Oliesta/doc-first-engineering, licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/).